ΨLM: Coupling Frozen Language and Physics Models through Trainable Latent Bridges

Ryoji Furui*

September 5, 2026

Abstract

We ask whether a pretrained language model and a pretrained physics model can run as one system — two frozen networks answering a single question with communication carried entirely by hidden states, the way the brain's hemispheres cooperate across the corpus callosum. As of August 2026 we find no published work coupling a pretrained LLM to a neural-operator physics model through a bidirectional latent channel at inference. We build such a system, ΨLM, on a consumer laptop (Apple M2, 24 GB), in four steps. (1) Tool-loop baselines reproduce the known capability-gap effect: a simulator adds +28.3 points to a 3B model on physics QA but *hurts* a 0.5B model. (2) We produce, to our knowledge, the first public reproduction of the Bicameral Model [12]: two frozen Qwen2.5-0.5B streams coupled by a 6.2M-parameter gated hidden-state interface reproduce the paper's phase transition, reaching 100% exact tool recall through the latent channel alone. (3) With the auxiliary language model replaced by a frozen Fourier Neural Operator, ΨLM answers Burgers-equation field-value questions at 100% (tolerance ±0.05; MAE 0.014) where the LLM alone achieves 5%, matching the accuracy of an oracle that states the answer in text — with no text at the interface. (4) The result survives hardening: held-out question families expose a clean generalization law (*support coverage* — readouts generalize only where their training support covers the test distribution, a hypothesis we confirm by refuting the alternative), and the architecture transfers to 2D reaction–diffusion with a pretrained physics foundation model (DPOT-Tiny) as the hemisphere, scoring 95%, within five points of the oracle ceiling. All experiments, including two failed designs and one refuted hypothesis, are reproducible from the public repository on a single consumer machine.

1 Introduction

Large language models reason in discrete tokens; physics foundation models [3, 17] evolve continuous fields. The human brain suggests a third architecture: specialized regions, neither subordinate to the other, co-processing a single task through dense internal communication. This paper takes the analogy literally. We couple a frozen pretrained LLM to a frozen pretrained physics model through small trainable *bridges* operating purely on hidden states — the language stream never sees the physics as text, and the physics model never parses a sentence — and ask whether the coupled system can answer questions neither component can answer alone.

A second motivation is epistemic. In the wave–particle picture of quantum mechanics, a discrete observation is one reading of an underlying continuous process; Bohr called the two descriptions *complementary*. ΨLM instantiates that structure operationally: a continuous field evolved by a

*`https://github.com/ryoji-info/PsiLM`. **AI generation disclosure:** the research reported here was designed, implemented, executed, and written up by *Claude Fable 5* (Anthropic), working autonomously on the author's Apple M2 laptop under the author's direction; see Section 12.

neural operator, and a discrete verbal reading of it produced by a language model, with a learned interface deciding what crosses between them. The name is the wave function Ψ.

Our literature survey (Section 2) found the ingredients of such a system published separately — frozen-model latent bridges [2, 45], two LLMs coupled in hidden-state lockstep [12], LLM–simulator loops through text [21, 39], and brain-inspired shared workspaces [14, 38] — but no published coupling of a pretrained LLM to a neural-operator physics model through a bidirectional latent channel at inference, a gap independently noted as open by a survey of latent inter-model communication [23]. We fill it at small scale, on consumer hardware, with the following contributions:

1. **Baselines** (Sec. 4): a seeded physics-QA benchmark on which text-level tool coupling reproduces the published capability-gap effect (+28.3 points at 3B, −3.3 at 0.5B).

2. **A reproduction** (Sec. 5): the first public reimplementation of the Bicameral Model [12], confirming its training phase transition and its causal order, and documenting two facts the paper does not: an inference bootstrap required at the uncoupled prompt boundary, and a teacher-forced/rollout dissociation showing exposure bias dominates evaluation at this scale.

3. **ΨLM** (Sec. 6): a frozen LLM and a frozen neural operator coupled through $\sim$3.5M trainable bridge parameters, reaching the oracle-text ceiling on 1D Burgers field-value QA (100% vs. 5% for the LLM alone).

4. **A generalization law** (Sec. 7): on held-out question families the interface transfers partially, failure is attributable to a single readout, and a controlled experiment refutes the natural "classify, don't regress" fix — establishing *support coverage* as the operative principle.

5. **Foundation-model transfer** (Sec. 8): the architecture carries to 2D reaction–diffusion with pretrained DPOT-Tiny [17] as the physics hemisphere (95% vs. 6.7% alone).

6. **Scaling the language hemisphere** (Sec. 9): a backbone-agnostic parameterization and an MLX port carry the architecture to Qwen3-1.7B (93.3% vs. a 96.7% oracle) and to 4-bit backbones; a trained revision loop adds +16.7 points on the held-out combination family; and at Qwen3-8B a probe-by-probe analysis across eight runs locates two scale-dependent failures — a learned attention pointer that never trains, and a field-carrying channel the frozen model shortcuts — and the system that removes both closes to within a few points of its oracle; a guard-rail benchmark then shows its gate open on every prompt (a 55-point loss on GSM8K with the bridges attached), and a no-harm training arm makes the gate selective — GSM8K and MMLU return to the backbone's numbers with the physics result retained (Tables 6 and 7); the finished recipe then transfers in one pass to a second model family, Gemma 4 12B (96.7% against a 98.3% oracle, gate selective), with one measured, backbone-specific fix to the readout normalization (Section 9.5).

2 Related Work

Text- and tool-level coupling. Mind's Eye [21] pioneered LLM reasoning grounded by a physics engine through prompt round-trips; RAP [16] framed reasoning as an agent conversing with a world model in text; Zephyrus [39] couples an LLM to numerical weather foundation models through code; SGA [25] alternates an LLM with differentiable simulation for scientific discovery; RIA [37] closes an LLM/world-model loop per decision step for driving. In all of these the physics side is a subroutine reached through text or code. Multiagent debate [9] couples multiple reasoner instances through exchanged messages; Cicero [10] combines a language model with a strategic-reasoning module through structured intents. Position papers argue that language, agent, and world models must interoperate [18, 19].

Latent-level coupling. Flamingo [1] and BLIP-2 [20] graft frozen perception onto frozen LLMs unidirectionally; CALM [2] cross-attends between two frozen LLMs' layers; GATS [45] exchanges activations bidirectionally among frozen models at different rates. Socratic Models [42] made language itself the inter-model protocol; a fast-moving line replaces it with embeddings, activations, or KV state [22, 28, 32, 34, 36, 40, 46]; DroidSpeak [24] shows even same-architecture models need repair to share raw state, motivating trained interfaces. Closest to our Stage 1, the Bicameral Model [12] runs two frozen LLMs in per-token lockstep through a trainable gated interface; no code was released.

Brain-inspired modular systems. The Global Workspace line [8, 14, 26, 38] couples frozen specialists through a learned shared latent space; bilateral architectures [31, 33] build literal two-hemisphere networks with an ablated callosal channel; Béna and Goodman [4] show module specialization requires a bandwidth-constrained channel. Dual-process agents [6, 7] and dual-system robot policies [5, 11] run a slow semantic model and a fast control model concurrently over latent interfaces.

Language and PDE operators. UPS [35] projects FNO features into a pretrained LLM's token space; Unisolver [44] conditions a neural solver on frozen-LLM equation embeddings; ICON-LM [41] fine-tunes a GPT-2 into an operator learner; Text2PDE [43] generates rollouts from text; Negrini et al. [27] decode operator states into text within one purpose-trained network. Each realizes one direction, or fuses everything into a single network; none couples two *pretrained* models bidirectionally at inference.

3 Experimental Platform

All experiments run on one Apple M2 laptop with 24 GB unified memory. The language hemisphere is frozen Qwen2.5-0.5B-Instruct [29] (hereafter Qwen2.5-0.5B; fp16 on PyTorch MPS; 4-bit MLX quantizations for Stage 0; Section 9 substitutes larger Qwen3 backbones), staged so a forward pass can pause at any decoder layer, exchange state, and resume; the staged pass is bit-exact against the stock model. Ground truth for the PDE tasks (Sections 6–8) comes from spectral solvers with integrating-factor RK4 (dt-convergence $\leq 4 \times 10^{-11}$; analytic limits verified), never from the neural physics models; Stages 0–1 are scored against closed-form mechanics and exact arithmetic. Trained components are only the interfaces (2–7M parameters on the 0.5B backbone; 12.6M and 42M for the larger backbones of Section 9); training runs in resumable 500–1000-step chunks at 1.8–12 s/step depending on the backbone.

4 Stage 0: Text-Level Tool Coupling

We construct a seeded 60-item benchmark of rigid-body comparison questions (five scene types; one third are physics traps such as equal-mass free fall and complementary launch angles) scored against closed-form mechanics, and evaluate an *alone* arm against a *tool* arm in which the LLM extracts scene parameters as JSON, a simulator computes the outcome, and the result returns to the prompt [21]. Parameter extraction succeeds on 100% of items at both model sizes, yet coupling helps only the larger model: Qwen2.5-3B improves 38.3% → 66.7% (+28.3, matching Mind's Eye's published +27.9 zero-shot average), while Qwen2.5-0.5B *degrades* 35.0% → 31.7% — it cannot reliably compare

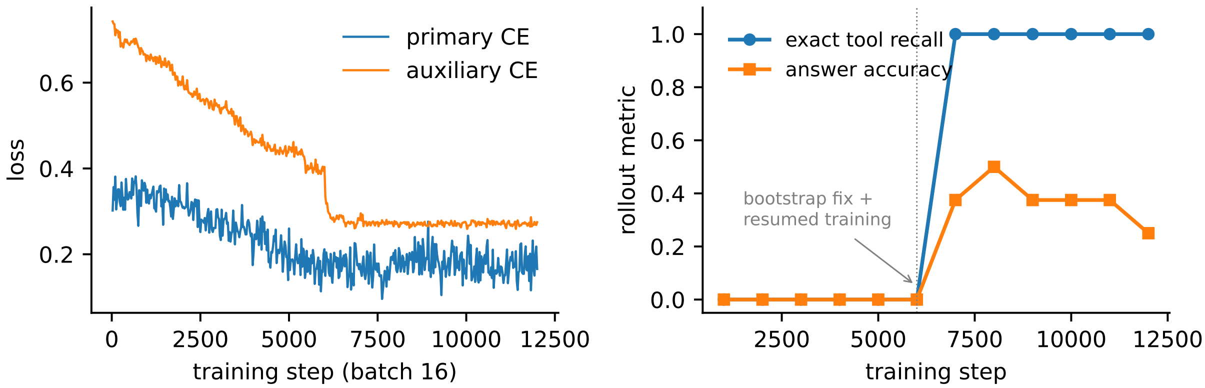

Figure 1: Stage 1 reproduction. **Left:** teacher-forced losses. **Right:** rollout metrics per 1000-step chunk. Exact tool recall undergoes the paper's phase transition ($0 \to 1.00$ within one chunk, at $\sim$112k samples) after an inference-bootstrap fix at step 6000; answer accuracy sets in after recall, in the paper's causal order.

two numbers handed to it in text. Evidence *use*, not extraction, is the bottleneck, motivating a trained interface that delivers evidence below the token level.

5 Stage 1: Reproducing the Bicameral Model

We reimplement the Bicameral Model [12] from its published specification: two frozen Qwen2.5-0.5B streams generate in lockstep; a trainable interface reads the primary's residual stream at layer 10 and writes a gated update into the auxiliary at layer 10 (forward coupling), and symmetrically auxiliary→primary at layer 15 (reverse), each update $h \leftarrow (1-\sigma)h + \sigma f(h_{\text{send}})$ with the suppression gate σ computed from the *receiver*. The auxiliary drives a calculator whose output is forced into the auxiliary stream only; the primary receives results exclusively through the latent channel. We train the 6.2M-parameter interface with dual-target SFT on procedurally generated multiplication with causality-aligned auxiliary traces (192k samples).

The phase transition reproduces, in its causal order (Fig. 1): forward coupling strengthens first, exact tool recall then jumps $0.00 \to 1.00$ in a single chunk, and answer accuracy sets in afterward. On held-out problems (7–10-digit products) the coupled system emits the exact `calc(A*B)` on 100% of rollouts — the auxiliary reads the operands entirely from hidden states — while the reverse channel carries the leading 4–6 result digits reliably and degrades toward the tail (digit similarity 0.689 vs. 0.371 for the frozen model alone; exact match only 2.5% at this length, $\sim$25–50% on the shorter mixed-length products of the per-chunk rollout evaluations, n=8 each).

Two findings absent from the original paper: (i) the first auxiliary token is predicted from the *uncoupled* prompt tail, a position training can never influence, so free-running generation derails without a protocol-level bootstrap (we force two initial wait tokens); (ii) high teacher-forced channel quality coexists with zero rollout success until that bootstrap exists — exposure bias dominates evaluation at this scale (the committed diagnostic at the final checkpoint, `teacher_forced_diagnostic.json`: 100% call-token accuracy and 63.0% per-digit, against 0% rollouts before the bootstrap).

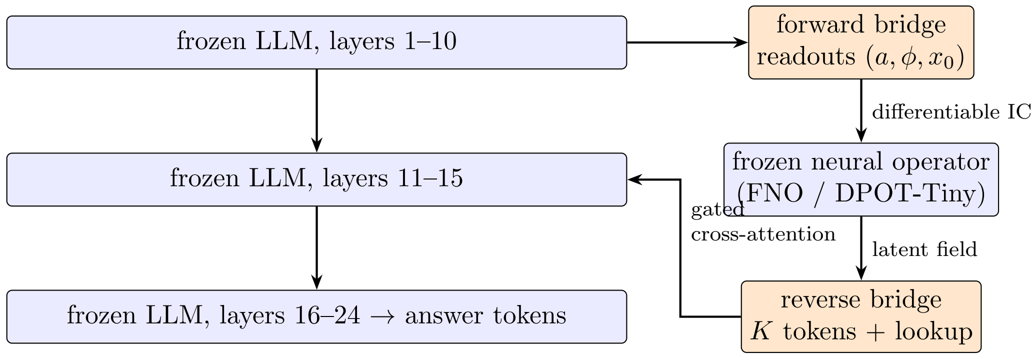

Figure 2: ΨLM. Orange components (3.5–3.7M parameters) are the only trained ones. The question is read out of the LLM's layer-10 hidden states; the frozen operator evolves the field; the result returns through a position-lookup token and gated cross-attention at layer 15. No text crosses the interface.

6 ΨLM: A Language Model Coupled to a Neural Operator

Stage 2 replaces the auxiliary language model with a genuine physics model (Fig. 2). The task: "$u(x, 0) = a\sin(2\pi x + \phi)$ evolves by Burgers' equation ($\nu = 0.02$) until $t = 0.5$; what is u at x_0?" — answered to two decimals, scored within ± 0.05 on fields spanning ± 0.6 (a degenerate always-zero strategy scores 7.9% on the training distribution and 1.7% on the held-out set of Table 1). The physics hemisphere is a 70K-parameter FNO trained once on solver data to 0.28% relative error (one-shot $u_0 \mapsto u(\cdot, T)$, in the full-rollout style of contemporary physics foundation models) and then frozen.

Forward bridge (language→physics). Learned queries attention-pool the LLM's layer-10 prompt states; heads regress $(a, \sin\phi, \cos\phi)$ and read the query position x_0 as a 100-way classification whose softmax expectation is differentiable and sharp. A differentiable sinusoid built from the readout feeds the operator, so answer-loss gradients flow from the LLM's words into the physics input.

Reverse bridge (physics→language). K=16 learned queries compress the operator's latent field (plus Fourier positional features and the predicted field itself) into soft tokens, joined by a *position-lookup token*: a periodic von Mises kernel of learnable sharpness, centered at the x_0 readout, samples the field "where the question asked," with an auxiliary head deep-supervised on $u(x_0)$. All tokens enter the primary at layer 15 through gated cross-attention (receiver-computed suppression gate, near-closed initialization).

Training minimizes answer cross-entropy plus auxiliary readout losses (parameter MSE, x_0 CE, lookup-value MSE); both backbone networks stay frozen. Two design elements were forced by failure analysis, which we report as results: plain cross-attention over field summaries mode-collapsed to per-trajectory constants (the system answered the field's *average* for every x_0), and a regressed x_0 (RMS error 0.06) was too blurry beside a shock; the classification pointer and the deep-supervised lookup cracked both: first-chunk rollout accuracy was 0.08 for both failed designs, then 0.50 and 1.00 in the final design's first two 16k-sample chunks (Fig. 3 shows the final configurations).

Result (Table 1, left): ΨLM answers 100% of held-out questions within tolerance (MAE 0.0138), matching the accuracy of an oracle that states the true value in the prompt — while the frozen LLM alone scores 5%. The answer travels entirely through hidden states: question out of the residual stream, simulation in the operator, value back through gated attention, verbalized by a frozen model.

Table 1: Held-out results. Left: 1D Burgers (n=60). Right: 2D Fisher–KPP with pretrained DPOT-Tiny (n=60). Tolerance ± 0.05; greedy decoding; all arms share the frozen Qwen2.5-0.5B.

	1D Burgers (FNO)		2D Fisher–KPP (DPOT-Tiny)	
arm	acc@0.05	MAE	acc@0.05	MAE
LLM alone	5.0%	0.729	6.7%	0.337
LLM + answer in text (oracle)	100%	0.0028	100%	0.0024
ΨLM (latent coupling)	**100%**	**0.0138**	**95.0%**	**0.0168**
degenerate always-0.00	1.7%	0.308	1.7%	0.673

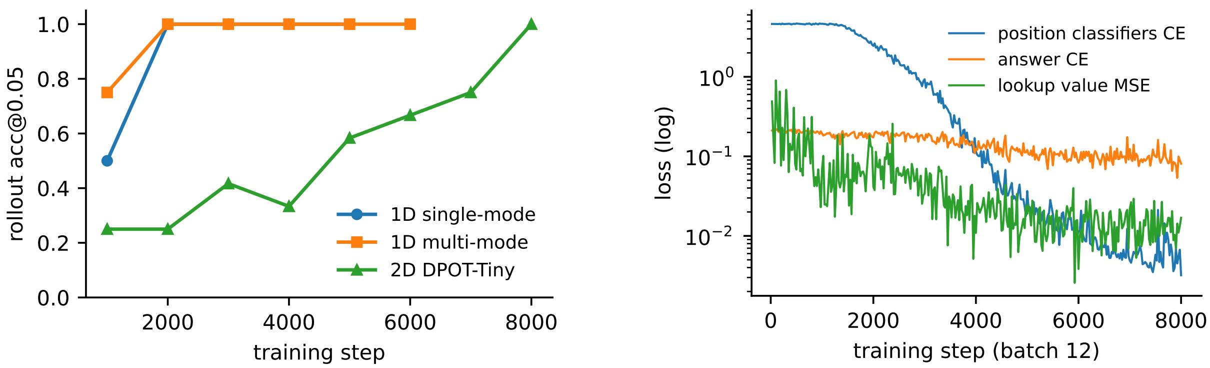

Figure 3: **Left:** rollout accuracy per training chunk for the three ΨLM configurations. **Right:** 2D training phase structure — the four positional classifiers lock in first (over ∼5k steps); the answer loss converges only after the pointer does, echoing Stage 1's causal ordering.

7 Generalization: the Support-Coverage Law

To harden the result we extend the initial conditions (ICs) to sums of sinusoids and train the bridges *only on single-mode questions*, evaluating on three families: in-distribution, the held-out combination (two-term questions never seen), and amplitude extrapolation beyond the training range. The physics FNO is trained on a broader distribution than any evaluation family, so transfer failures are attributable to the interface. Results (n=48 per family): in-distribution 97.9% (MAE 0.009); unseen combination 31.3% (0.128); extrapolation 50.0% (0.096) — all above the LLM-alone (0–6.3%) and always-zero (27.1%, 16.7%, 4.2%) baselines, decisively so except against the combination family's zero baseline, and all far below in-distribution.

Teacher-forced attribution is unambiguous: the x_0 pointer generalizes *perfectly* to every family (absolute error $\leq 10^{-4}$), while the amplitude readout alone degrades $0.043 \rightarrow 0.355$ (combination) and 0.206 (extrapolation); the record is committed as `attribution.json`. The natural hypothesis — the classified quantity transfers, so classify the amplitudes too — we tested with a v2 interface (per-mode pooling queries; amplitudes as 121-bin classifications). **It was worse on every family** (extrapolation 50%→19%, combination 31%→21%; v2 was evaluated at its 64k-sample convergence plateau — rollout accuracy flat across its final two chunks — vs. v1's 96k). The refutation identifies the real principle: x_0 generalized because all 100 of its bins occur in training, whereas amplitude bins outside the trained range never receive gradient and cannot move, while regression at least drifts. We summarize this as a *support-coverage law*: a trained latent interface generalizes like a learned model, not a program — coverage of the readout's support at training time, not the readout's parameterization, is what buys transfer. Section 9.6 sharpens it at 12B into two coverages that can be bought separately — structure and value — and reports which of the two the refuted v2 interface was missing. Section 8 applies the law prospectively.

8 2D Physics with a Pretrained Foundation Model

Finally we replace our own operator with DPOT-Tiny [17]: a 7.5M-parameter AFNO operator pretrained across twelve PDE datasets, loaded with `strict=True` from the published checkpoint, fine-tuned for 1200 steps (≈5 minutes) to 1.64% relative error on our task, then frozen. The task moves to 2D Fisher–KPP reaction–diffusion on the periodic unit square: a Gaussian bump — height, center, and width all stated in the question — grows and spreads by $u_t = D\nabla^2 u + r\,u(1-u)$, and the question asks for u at a point (x_0, y_0) sampled within the front's active zone (always-zero scores 4.8% on the training distribution; 1.7% held-out). The IC is replicated across DPOT's 10-timestep input history; the reverse bridge attends over DPOT's 256 latent patch tokens plus a separable 2D periodic lookup on the predicted field; applying the support-coverage law, we make every positional readout (bump center and query point) a fully covered 100-bin classification.

Training exhibits a clean phase structure (Fig. 3, right): the four positional classifiers lock in over ∼5k steps (CE 4.6 → 0.04), and the answer loss converges only after the 2D pointer does; rollout accuracy climbs 0.25 → 1.00 over 96k samples. Held-out (Table 1, right): **95.0%** at MAE 0.0168, within five points of the oracle-text ceiling, against 6.7% for the LLM alone. The 1D result survives both moves — to 2D, and to a physics hemisphere that is a genuine pretrained foundation model.

The same task at 12B. Rerunning stage 2d on Gemma 4 12B (the recipe of Section 9.5, 7,500 steps: 2,000 readout-only, 4,000 coupled, 1,500 no-harm; `results/stage2d_gemma12b_2d/`) gives **100%** at MAE 0.0096 on sixty held-out questions, against 10.0% for the backbone alone and **96.7% for the oracle** — the only arm in this work where the latent channel beats the text ceiling. The mechanism is mechanical rather than mysterious: the oracle must copy a number out of the prompt and occasionally mis-rounds it (MAE 0.021), while the bridge reads the field value exactly and the answer never passes through text (0.0096). The coupled phase ended at 93.8% and the no-harm phase ran 97.9%, 95.8%, 100%, so gate selectivity again cost nothing on the task. One engineering note for hybrid stacks: MLX and torch share one unified GPU memory and MLX's cached buffers are invisible to torch's MPS allocator, so the no-harm arm — whose prompts are the long ones — died asking for 256 bytes with 42 GiB in "other allocations"; returning the cache at the boundary was not sufficient and the physics model (7.5M parameters) simply runs on the CPU, at 65 ms against 48 ms per batch-4 call.

9 Scaling the Language Hemisphere

Stage 0 predicts that the payoff of coupling grows with the language model's ability to *use* evidence. This section reports what happened when the language hemisphere was scaled from 0.5B to 1.7B and 8B parameters with the physics hemisphere (the Stage-2 FNO, byte-identical) and the task held fixed. The 1.7B result is a clean transfer. The 8B was not, and we report it as a scale-dependent failure analysis in the spirit of Sections 6 and 7: each step isolates one component, and the sequence ends in a decision, a concession, a system that works on its task for a reason we can state, a guard-rail measurement of what it cost off-task, and the training arm that removed the cost.

Table 2: Loop-coupling ablation on the multi-mode task of Section 7 (ΨLM arm; n=48 per family; acc@0.05 / MAE; both trained arms at 96k samples).

family	single pass	loop-trained	inference-only loop
in-distribution	97.9% / 0.009	**100%** / 0.007	8.3% / 0.323
unseen combination	31.3% / 0.128	**47.9%** / 0.089	0.0% / 0.868
amplitude extrapolation	**50.0%** / 0.096	47.9% / **0.055**	6.3% / 0.416

9.1 Backbone parameterization and Qwen3-1.7B

The interface is parameterized by the backbone's configuration alone: coupling depths are the Stage-2 fractions of depth (read at 10/24, inject at 15/24, rounded), bridge widths follow the hidden size, and the staged forward pass is verified bit-exact against the stock model for every new backbone before training. On Qwen3-1.7B [30] (28 layers, hidden 2048; fp16 on PyTorch MPS; thinking mode disabled) the bridges resize to 12.6M parameters, couple at layers 12/18, and train at 5.2 s/step. Under the Stage-2 budget (5000 steps $\times$ 16 = 80k samples) the per-chunk rollout probe reaches 1.00 in the fifth chunk, and the held-out evaluation (n=60, the set of Table 1) gives **93.3%** at MAE 0.022 against an oracle-text ceiling of 96.7% (MAE 0.021), with the LLM alone at 1.7% (MAE 2.57) and the always-zero strategy at 1.7%. Two observations. First, the receiver-computed gate, which settles at $\sigma \approx 0.13$ on the 0.5B, opens fully ($\sigma \to 1.0$) within the first 25 steps on the 1.7B and stays there: the stronger backbone takes the channel at full strength. Second, an evaluation-protocol lesson: the terse "reply with only the number" instruction that worked at 0.5B made Qwen3-1.7B's non-thinking mode drop minus signs systematically (oracle 47%, half the negatives wrong, while free-form answers were correct). All text arms now use an "`Answer: <number>`" marker with marker-aware parsing, under which the coupled and oracle arms of Table 1 reproduce (100%/0.0138 and 100%).

9.2 Loop coupling: revision buys composition

Before scaling the backbone we asked whether the interface itself had headroom on the Section 7 families. `loop_model.py` runs two read→rollout→inject passes ordered $r_1 < w_1 < r_2 < w_2$ (layers 8/11, then 15/18 of 24), so the second readout sees the stream *after* the first injection and can revise it; the bridges are shared across passes (a recurrence), every pass's readouts are supervised, and the answer loss is applied once. Table 2 compares it with the single-pass interface at a matched 96k-sample budget, plus a third arm that simply runs the single-pass-trained bridges as two passes at inference. **The trained loop buys compositional generalization**: +16.7 points on the unseen combination family, with MAE improved on every family (the extrapolation error nearly halves at flat accuracy — support coverage still rules there). The third arm collapses: switching the loop on at run time without training it is catastrophic, not neutral, with the caveat that this arm changes the coupling depths and the untrained revision at once, so it bounds the zero-shot transplant rather than isolating the revision effect. Loops must be trained in; they then pay off exactly where the single-pass interface was weakest.

9.3 The MLX port

Backbones beyond ~2B do not fit 24 GB in fp16; their 4-bit quantizations do, so we ported the machinery to MLX [15]. The port (`psilm/mlx/`) is a segmented forward over `mlx-lm` models with explicit causal and padding masks, bit-exact against the stock model on the 4-bit Qwen2.5-0.5B (maximum logit difference 0.0, greedy tokens identical); an FNO port carrying the complex spectral

Table 3: Copy probe: a random value $v \sim U(-1, 1)$ injected through the gated cross-attention alone (see text). 600 steps $\times$ batch 8; greedy rollouts, n=24, tolerance ± 0.05; σ is the mean gate at the end of training.

backbone (4-bit)	layers / width	inject at	bridge	s/step	final σ	acc@0.05	MAE
Qwen2.5-0.5B	24 / 896	15	1.49M	0.36	0.096	100%	0.0075
Qwen3-8B	36 / 4096	22	8.41M	5.55	0.042	91.7%	0.025

weights as (real, imaginary) pairs, converted exactly from the trained PyTorch checkpoint (field difference 10^{-7}); and fp32 bridges trained by `nn.value_and_grad` through the frozen quantized backbone. Validation on the 4-bit 0.5B under the identical task and budget: per-chunk rollouts reach 1.00 at MAE 0.0136 by step 5000 (n=12 probe), against the fp16 run's 1.00/0.0138 — the bridges absorb the backbone's quantization noise — at 1.9 s/step against 2.3 for fp16 PyTorch.

9.4 Qwen3-8B: a scale-dependent failure analysis, and what it took

Qwen3-8B-4bit (36 layers, hidden 4096) is the largest dense backbone that trains on this machine (12 s/step at batch 8; bridges 42.0M, most of it the 4096×4096 pooling keys). Its staged forward passes the parity test. What did not carry over was training, and the failure moved as we fixed it. We summarize eight runs; runs 1–4 are recorded in the commit history, runs 5–8 in `results/stage2_mlx8b4/`, `_mlx8b6/`, `_mlx8b7/` and `_mlx8b8/`.

Saturation. At 4096 dimensions the raw residual-stream magnitudes saturated the bridges' pooling softmax and gates: the x_0 classifier sat exactly at uniform cross-entropy ($\ln 100 = 4.61$) for 8k samples while the gate drifted to $\sigma = 0.86$ — no gradient reached the pointer (the very first unclipped setup steps had produced losses of 56 and 362). The fix is to make the interface scale-free: parameter-free RMS normalization of every bridge *input* (pooling, gate, and injection queries; the residual stream itself is untouched), and an injection scaled by the receiver's local stream magnitude, $h \leftarrow h + \sigma\, \mathrm{RMS}(h)\, f(\cdot)$.

Flatness, then a closed gate. RMS normalization traded saturation for flatness: the learned x_0 pool now averaged the whole prompt, and the classifier sat at uniform CE for a full 2500-step run while the parameter readout and the gate were healthy — the pointer was starved, not the channel. Supervising the pool's attention mass on the x_0 token span (which the QA builder can compute exactly, since it wrote the prompt) moved that mass from 6% to 98.5% in six steps, and in the following run the pointer, parameters, and lookup all trained (pointer error 0.014; lookup MSE 0.001) — yet rollouts stayed at 0–25% and the gate closed ($0.12 \to 0.065$). The mechanism is specific to strong backbones: their answer cross-entropy starts at its digit-entropy floor because the format tokens are already solved, so early injection is pure noise, the receiver-computed gate suppresses it, and the channel never receives gradient. Digit-weighted CE ($\times 5$ on digit tokens), an open gate initialization ($\sigma_0 = 0.5$), and injection at 75% depth (layer 27) did not rescue the fourth run. Two hypotheses were then live: (H1) large frozen backbones resist latent steering — the *channel* is the problem; (H2) the forward *readout* is the problem. We separated them with two probes.

Copy probe (H1). The minimal coupling task: v is encoded by a small MLP into eight soft tokens and injected through the same gated cross-attention into a prompt ("What is the secret number?") that carries no information, so the only route from v to the answer is the channel. Table 3: the 8B verbalizes the injected number at 92% (MAE 0.025) after 600 steps, about 55 minutes, against 100% for the 0.5B — and does so with a gate of $\sigma = 0.04$. **H1 is false**, and the low

Table 4: Readout probe: the forward readout in isolation, trained only to classify x_0 into 100 bins from the RMS-normalized prompt states at the coupling layer (10/24 for the 0.5B, 15/36 for the 8B). Held-out exact-bin accuracy and mean absolute error (n=48); a uniform-random guess scores ≈ 0.33 on the error. *span* = parameter-free mean over the builder's x_0 token span (oracle pooling).

backbone (4-bit)	pooling	steps × batch	exact bin	$\lvert \hat{x}_0 - x_0 \rvert$
Qwen2.5-0.5B	learned query	400 × 8	2.1%	0.328
Qwen2.5-0.5B	span	400 × 8	18.8%	0.064
Qwen3-8B	learned query	400 × 8	2.1%	0.247
Qwen3-8B	learned query + attention supervision	400 × 8	2.1%	0.274
Qwen3-8B	final prompt position	400 × 8	0.0%	0.378
Qwen3-8B	span	400 × 8	2.1%	0.099
Qwen3-8B	learned query	2000 × 16	0.0%	0.284
Qwen3-8B	span	2000 × 16	**83.3%**	**0.0046**

gate values seen throughout were never suppression: 0.04 is the scale at which a 4096-dimensional stream wants the signal.

Readout probe (H2). We then isolated the forward readout — no physics, no injection, no answer loss — training only a pooled readout over the coupling-layer prompt states to classify x_0 (Table 4), with four ways of pooling: the learned query of the current design; the same plus attention-mass supervision; the final prompt position alone; and *span pooling*, a parameter-free mean over the builder's x_0 token span, i.e. the pooling an oracle pointer would produce. At the short budget the ordering is already clear — span $\gg$ learned $>$ final position, the last at chance, so neither model aggregates x_0 into its final prompt position and pooling over the span is necessary — but the 100-way classifier is under-powered there: the 0.5B's learned-query row, 2% exact here, is the same pointer that reaches 100% end-to-end in Section 6. At five times the budget the picture is unambiguous. **The learned pointer never leaves uniform at 8B**: its cross-entropy stays between 4.58 and 4.63, i.e. $\ln 100$, for all 2000 steps (0% exact), while span pooling over the identical hidden states reaches 83% exact-bin accuracy at error 0.0046. The information is present at the x_0 positions and readable by a two-layer head; learning *where to look* is what fails at this width, and it does not fail slowly — it fails at every budget we could afford.

Decision, and its cost. The readout therefore pools deterministically over the builder's span (the learned path remains as the fallback when no span is supplied). This is a real concession and we state it plainly: the builder knows where x_0 is because it wrote the prompt, so at 8B the language→physics *pointer* is supplied by the task rather than learned from the words, and a free-text question would have no such span.[1] What remains learned on the forward side is the readout of the value at those positions; on the reverse side, everything. Deterministic where the task permits, learned where it must be — the same principle as the position-lookup token of Section 6, applied one level further down.

Run 5. With span pooling, per-module gradient clipping (each bridge clipped at 1.0 separately, so the 4096-dimensional injection gradient cannot consume the global budget — the regime in which the probe converged), and the x_0 loss weight raised from 0.3 to 1.0 after the first chunk, the system trained for 2500 steps at batch 8: 20k samples, a quarter of the 0.5B budget. The parameter readout converged in the first chunk (MSE 0.46 → 0.02) and the digit-weighted answer loss fell

[1]The span is found by a token-sublist match on the last number in the prompt, widened by one token on each side; on a failed match the builder falls back, silently, to pooling the whole prompt. The fallback never triggered on the present data, but it should raise rather than degrade.

from 0.40 to 0.33, but the x_0 head stayed at CE 2.1–3.4 (pointer error 0.04–0.20) where the probe, with the identical head, pooling, and backbone, had reached 0.74 at 32k samples; and the gate fell from its open initialization to 0.29 by the end of chunk 1, to 0.11 by chunk 2, and to 0.03–0.04 from step ∼1100 onward, never reopening. Per-chunk held-out rollouts (n=12): 25%, 58%, 33%, 58%, **67%** (MAE 0.387, 0.108, 0.132, 0.074, 0.167). On the four-arm held-out evaluation (n=60; `results/stage2_mlx8b4/final_eval_summary.json`) the coupled system reaches **41.7%** (MAE 0.183) against 6.7% (MAE 0.706) for the backbone alone and 100% (MAE 0.0026) for the oracle — a six-fold gain over the uncoupled model on an eight-billion-parameter frozen backbone, and the first 8B run that learned end to end, but well short of the 0.5B and 1.7B systems, which close to within three points of their oracles. (The text arms of an 8B need a 768-token budget: Qwen3-8B writes a derivation before its `Answer:` line, and at 160 tokens every item truncated; even at 768, only 35% of the uncoupled model's replies reach the line, against 100% for the oracle and for ΨLM.)

Two causes, found by code review after the run and quantified against its log, were removed in run 6: the MLX trainer had built a fresh AdamW at every 500-step resume and, with the library's default of no bias correction, the first ∼15 steps after each resume took effectively 2–4× larger steps (both gate collapses and the answer-loss spike in the log coincide with resumes; the PyTorch trainer persists optimizer state); and the x_0 estimate fed the reverse bridge's lookup kernel without a stop-gradient, so the pointer head also received lookup and answer gradients, near-orthogonal to its own and about 0.4× its norm, the one structural difference from the probe in which the same head converged three times faster.

Runs 6–8: the channel's form. Run 6 persisted the optimizer with bias correction, detached the pointer, and added a readout-only warm-up (1000 steps through layers 0–15 only, at a third of the coupled cost) so that the pointer locks before the channel first opens. It settled every question about the pipeline's parts and failed anyway: the pointer reached error 0.002 with 100% exact bins, the lookup at the predicted position was correct (MSE 0.003), the gate stayed open at 0.20–0.27 at the answer positions, the injection ran at 30–50% of the residual stream — and the 48-item rollouts scored 21% and 17% with MAE 0.43, worse than answering zero. Its generations explain why (Table 5): every answer on a trajectory is the same number regardless of x_0 (−0.53 for all questions with a=1.28, −0.33 for a=0.60). The frozen 8B reads the amplitude, from the prompt text or from the sixteen field tokens, and ignores the one token among seventeen that depends on x_0. This is the mode collapse of Section 6, which the 0.5B escaped after some 80k samples; an 8B at 12 s/step affords a quarter of that. Run 7 capped the injection at 20% of the stream and re-initialized the channel at a conservative gate: within fifty steps the gate saturated at 1.0 and the injection at its cap, and the answer loss sat at 0.40 — exactly where it had sat in runs 5 and 6 at gates of 0.03 and 0.25. Under Adam the gate and the injection magnitude are bang-bang controls that saturate in whichever direction the first gradients point; the answer loss was indifferent to all of it. (A gate saturated at 1.0 on physics prompts is, as the guard-rail below shows, saturated on every prompt.)

Run 8 changed the channel's *form*. The copy probe had shown what an 8B reads: a clean encoding of one value. So the reverse channel became the copy probe's encoder — the deep-supervised lookup $\hat{u}(\hat{x}_0)$, detached, through Fourier features into eight soft tokens — injected at the copy probe's layer (22 of 36) under the same cap, with run 6's trained pointer and lookup kept. The channel can now carry only what depends on x_0. From the same checkpoint the answer loss moved for the first time in any 8B run (0.41 to 0.25 within the first chunk), and the 48-item rollouts went 89.6%, 91.7%, 95.8%, 100%, 97.9% over five chunks (MAE 0.025 → 0.014); on the sixty-item held-out set the coupled 8B reaches **98.3%** (MAE 0.0135; `results/stage2_mlx8b8/final_eval_summary.json`) against 6.7% for the backbone alone and 100% (MAE 0.0026) for the oracle — level with the 0.5B system's ceiling and above the 1.7B's 93.3%, on 20k coupled samples.

Table 5: The 8B runs from the deterministic pointer onward. Pointer = $|\hat{x}_0 - x_0|$ at the end of the run; gate and injection are measured at the answer positions; accuracy is acc@0.05 on 48 held-out items at the last chunk (run 5: n=60).

run	channel	pointer	gate / injection	acc	outcome
5	lookup + 16 field tokens	0.13	0.03 / ∼5%	41.7%	first end-to-end learning
6	same, warm-up + fixed trainer	0.002	0.25 / 30–50%	17%	per-trajectory constants
7	same, cap 0.2, re-initialized	0.003	1.00 / 20%	—	loss flat; stopped
8	value tokens (copy-probe form)	0.000	1.00 / 20%	**97.9%** (n=60: 98.3%)	reads the value, on every prompt

Table 6: Guard-rail benchmark on the run-8 system (`results/bench/v8_8b_guardrail_summary.json`). Same backbone weights and prompts in every arm; greedy decoding; n=100 per dataset. *zeroed* runs the full coupled pipeline with the injection multiplied by 0 and the gate still measured. parse = fraction of replies with a scorable answer (an unparsed reply scores wrong); $\bar{\sigma}$ = mean gate over positions, open = items whose mean gate exceeds 0.1. Burgers items are the first 100 of `data/stage2_qa_val.json`, not the 60-item held-out set of the text; the backbone's Burgers arm uses the nudge protocol at a 160-token budget, at which every reply truncates (6.7% at 768 on the 60-item set, `results/stage2_mlx8b8/final_eval_summary.json`), and the zeroed arm under the trained prompt produces no scorable reply.

dataset	protocol / budget	backbone	parse	zeroed	ΨLM	parse	$\bar{\sigma}$ (open)
Burgers QA	trained reply / 32	2%	100%	0%	**97%**	100%	0.99 (100%)
MMLU, 5 subjects	`Answer: <letter>` / 24	55%	66%	55%	69%	100%	0.99 (100%)
items the backbone answered (n=66)		83%	—	83%	85%	—	
GSM8K	`Answer: <number>` / 384	89%	100%	89%	**34%**	100%	0.98 (100%)

The lesson is the one the Bicameral Model and Béna & Goodman state for interfaces between modules: the channel must be narrow. At 0.5B a channel carrying the whole field could be trained to attend to the right token; at 8B, with a budget an order of magnitude smaller in samples and a frozen model with a far stronger prior, the channel has to carry the answer and nothing that lets the language side shortcut it. The 8B system that works is, in the end, the copy probe with the physics model in front of it — which is also the cleanest statement of what ΨLM is, and, as the guard-rail below shows, of what it still lacked at that point: a switch.

Guard-rail: the gate is not selective. A system that reads a physics model through a latent channel must also be run on prompts where there is nothing to read. `eval/bench_guardrail.py` does this with three arms on the same backbone weights, the same prompts, and greedy decoding: the frozen backbone alone; ΨLM with the run-8 bridges; and a *zeroed* arm that runs the full coupled pipeline — readout, FNO, value tokens — but multiplies the injection by zero while still recording the gate, so that zeroed vs. backbone is the cost of attaching the bridges and ΨLM vs. zeroed is the effect of the injection. Non-physics prompts carry no x_0 span, so the readout pools the whole prompt, which is exactly the builder's fallback noted above. Table 6 gives the result. The gate opens on every prompt of every dataset ($\sigma = 0.98$–0.99; 100% of items above the 0.1 threshold on GSM8K as on Burgers), the FNO returns a field value for whatever initial condition the readout extracts from a word problem ($\hat{u} \in [-0.34, 0.22]$ on GSM8K), and the value tokens carry it in. On GSM8K the coupled backbone falls from 89% to 34% (-55 points; 56 items lost, one gained; McNemar $p < 10^{-3}$). On 37 of the 100 items it emits a bare `Answer:` line with no derivation — two of them correct, 26 of them negative, as the injected field value was on every one — and on the 63 where it still reasons it is worse (32 correct against the backbone's 58). The MMLU column is a gain in appearance only. Under the benchmark's 24-token budget the backbone begins a derivation on every high-school-mathematics item and on 14 of 20 college-physics items and never reaches its `Answer:` line (parse rate 66%; an unparsed reply scores as wrong), while the injected model answers tersely and parses at 100%. On the 66 items both arms answer they score 55 and 56; the remaining 13 points come from items the backbone never answered, on which the coupled model scores 38% against a 25% chance floor. That is a length effect, not evidence use, and we do not count it. The zeroed arm is the control that locates the damage: its replies are byte-identical to the backbone's on all 200 non-physics items, so the staged forward with the bridges attached, the readouts, and the FNO cost nothing; every point lost is the injection. The cause is in the training data. The gate was trained on physics prompts only, under Adam it saturated at 1.0 in run 7 and stayed there (Table 5), and a gate that has never seen a prompt on which it should close is not a gate: at 8B the word "gated" in gated cross-attention has so far described a form, not a behaviour.

Run 9: a gate that closes. The missing arm is a *no-harm* arm: non-physics prompts from the GSM8K *train* and MMLU *validation* splits (1049 of them — 700 from GSM8K, 349 from MMLU — disjoint from every benchmark test item by normalized text, 300 of the GSM8K prompts stripped of the benchmark's `Answer:` line so that the gate cannot learn the wrapper), each paired with the frozen backbone's own greedy continuation of up to 32 tokens, alternated one-to-one with physics batches from the run-8 checkpoint. The coupled model is scored on them by a plain cross-entropy through the full pipeline plus a penalty on the mean gate, and on these steps only the gate's two layers receive gradients, so the one route down is to close. It closed within fifty no-harm steps: the gate at the answer positions, 0.55 over the first dozen no-harm steps, was below 0.001 by the fiftieth and 0.000 for the last half of the run, while settling at 0.94 on physics batches, with the physics answer loss unchanged (0.21–0.27 against run 8's 0.25). The same benchmark on the run-9 bridges (Table 7) shows GSM8K at 89% in all three arms (paired: 88 items correct in both, one swapped each way), MMLU at a 256-token budget within one item of the backbone and at the same 83.3% on the 72 items both arms answer, and a mean gate below 0.003 on both non-physics sets (no item above the 0.1 threshold; the largest per-item mean 0.054); on the physics set the gate is open on every item at 0.79 and the coupled backbone answers 95% (MAE 0.022), two items below run 8's 97% on the same hundred questions; on the sixty-item held-out set of Section 9.4 it scores 95.0% (MAE 0.021; `results/stage2_mlx8b9/final_eval_summary.json`) against run 8's 98.3%, the same two-item cost. GSM8K *without* the `Answer:` line — the form of the 300 no-harm prompts that dropped it, and one the physics arm never uses — runs at 79% in all three arms with the gate at 0.001, so what the gate learned is not the template. The coupling is selective, not merely harmless: the gate chose the mechanism the training arm named, and a system that reads a physics model on physics questions and is byte-for-byte the backbone everywhere else is, for the first time in this work, what the word "gated" was meant to describe. The text-only baseline for this backbone, rerun under the forced-answer protocol of Section 9.5 (n=60, `results/stage2_mlx8b9/final_eval_baseline_forced.json`), is 0/60 — every item answered, a systematic overestimate (mean +0.79 against truths at −0.03, MAE 0.89); the 6.7% of the unforced protocol was the parser reading numbers out of unfinished derivations. Under a deliberately generous protocol — four solved examples, 1,536 tokens, thinking enabled — it reaches 2/60 (3.3%), the answers echoing the examples' values; the best single constant would score 13.3%.

9.5 A second family: Gemma 4 12B

The recipe that closed the 8B campaign — deterministic span pointer, value-token channel, readout warm-up, no-harm arm — was then run once, as a single pass, on a backbone from a different family: Gemma 4 12B [13] (48 layers, hidden 3840, forty sliding-window and eight global-attention layers, a 262k tied vocabulary, and a tanh soft-cap on the logits), in its 4-bit MLX conversion. The staged

Table 7: Guard-rail benchmark on the run-9 system (`results/bench/v9_8b_guardrail_summary.json`; same protocol as Table 6 except a 256-token MMLU budget and a 768-token budget for the backbone's Burgers arm). The last row is the wrapper-cue probe (`v9_8b_nonudge_guardrail_summary.json`): GSM8K prompts without the `Answer:` line, the form of the 300 no-harm prompts that dropped it.

dataset	protocol / budget	backbone	zeroed	ΨLM	parse (both)	$\bar{\sigma}$ (open)
Burgers QA	trained reply / 32	5%	0%	**95%**	100%	0.79 (100%)
MMLU, 5 subjects	`Answer: <letter>` / 256	60%	60%	61%	72%	0.003 (0%)
items both arms answered (n=72)		83.3%	—	83.3%	—	
GSM8K	`Answer: <number>` / 384	89%	89%	**89%**	100%	0.002 (0%)
GSM8K, no `Answer:` line	free reply / 384	79%	79%	79%	100%	0.001 (0%)

forward is bit-exact against the stock model once the adapter unpacks the per-layer state, scales the embeddings, and applies the soft-cap; a batch-4 training step takes 12 s at a 13 GB peak (batch 8 sits at the working set), so the budget was matched by halving the batch and doubling the steps.

The one backbone-specific fix. The first pass plateaued at 40–42% on the 48-item rollouts where the 8B had reached 90% at the same sample count, and the phase means placed the fault in the pointer: at cross-entropy 1.8 and 47% exact bins, the value looked up at the predicted position carried an RMS error near 0.1, which no channel can turn into answers within 0.05. The cause is a property of the residual stream. Both backbones carry massive-activation dimensions (their top five dimensions hold 95% of Gemma's energy at the readout layer and 99.9% of Qwen's), but Gemma's are nearly constant across prompts, so the per-position RMS normalization of the bridge input divides the digit signal by them: the across-item variance of the pooled x_0-span vector, which is all the classifier has to work with, is 26 times smaller on Gemma than on Qwen (0.99 against 26.0). Standardizing each dimension with mean and standard deviation from a 32-prompt calibration pass at the readout layer — two frozen vectors stored with the bridges — restores it to 51.8 (`results/readout_variance/`; the same standardization also doubles Qwen's, to 52.7, which we have not yet exploited), and the warm-up then ends with the best pointer of any backbone in this work (cross-entropy 1.24, error 0.009, 69% exact bins; the 8B: 1.76, 0.019, 53%).

Result. On the sixty-item held-out set (`results/stage2_gemma12b/final_eval.json`) the coupled Gemma reaches **96.7%** (MAE 0.017) against 98.3% (MAE 0.007) for the oracle and 0% for the backbone alone, which never reaches an `Answer:` line within 768 tokens (started on an `Answer:` line and forced to commit, it scores 6.7% at MAE 0.42 — four near-constant guesses, 0.41/0.51/0.54/0.11, inside the tolerance; the best single constant would score 13.3%; `final_eval_baseline_forced.json`). The no-harm phase, run at a third of the learning rate after the coupled phase's per-chunk scores had begun to swing by thirty points at batch 4, closed the gate on the negatives (0.01 at the answer positions against 0.17 on physics prompts) while the physics rollouts went 100%, 100%, 97.9%. Gemma's gate settles near 0.15–0.19 with the injection at 3–4% of the residual stream, a far gentler operating point than the Qwen 8B's saturated gate at the 20% cap; both read the value. The guard-rail of Table 7, rerun on the Gemma bridges (`results/bench/gemma12b_guardrail_summary.json`), gives 97% on the physics set with the gate open on every item at 0.14, and on the non-physics sets the backbone's numbers to within noise with the gate shut on every item: GSM8K 84% in all three arms (paired: 82 items correct in both, two swapped each way; gate 0.004), MMLU at 256 tokens 53%/55%/53% and identical at 79.1% on the 67 items both arms answer (gate 0.008), and GSM8K without the `Answer:` line 83% in all three arms (gate 0.002). One difference from the Qwen runs is worth recording: with the injection zeroed, the trained Gemma prompt still scores 10% on the physics set where the Qwen zeroed arm scored 0%, so a tenth of the Gemma answers are recoverable from the reply template alone; the coupled arm's 97% is against that floor, not against zero. The recipe, then, transfers across families with one backbone-specific adjustment, and the adjustment is a measured property of the residual stream rather than a tuned hyperparameter.

Mixture-of-experts verdict. The natural next backbone on 24 GB is a sparse one. Qwen3-30B-A3B-4bit [30] (48 layers, hidden 2048, 128 experts, 8 active) loads in 17.2 GB and trains at a 19.8 GB peak, and the staged forward is bit-exact through its MoE blocks. One blocker had to be fixed: `mlx-lm`'s MoE block feeds the router's `argpartition` indices straight into the expert gather, so autograd raises on the vector–Jacobian product; `moe_patch.py` wraps the routing indices in `stop_gradient`, which is also the mathematically correct statement (discrete selections carry no gradient; the router's softmax scores and the expert paths still do), with parity unchanged. The verdict is on cost: 97.8 s/step at batch 4 — roughly 24 s per sample against 1.5 for the dense 8B — because sparsity saves FLOPs, not the gather/scatter dispatch that dominates on Metal. A MoE backbone is feasible here but not economical, and dense 8B remains the scaling vehicle. We note one measurement it uniquely affords: with untrained bridges, the injection already changes the top-8 expert selection at 9.3% of positions at the coupling layer, so routing steering by the physics hemisphere is directly observable.

What the campaign established. The architecture transfers to 1.7B within three points of its oracle, and a trained revision loop extends what the 0.5B interface can compose. At 8B the latent channel works (copy probe), the readout's content is present and linearly readable (span probe), the learned attention pointer is dead at any budget — a scale-dependent failure of *optimization*, not of capacity or of the frozen model's willingness to be steered — and a channel that carries the whole field is shortcut by a frozen model strong enough to read the amplitude from the prompt. Four interface changes survive from the campaign as general lessons: scale-free bridges, digit-weighted answer loss for backbones that have already solved the format, deterministic pointers wherever the task can supply them, and a physics→language channel that carries the queried quantity and nothing the language side could use to avoid it. With those, an eight-billion-parameter frozen backbone reads a frozen physics model through a latent channel and answers within a few points of the oracle, on the same budget-per-sample the 0.5B needed to reach its ceiling. It first read the channel where there was nothing to read — the gate was open on every prompt and GSM8K fell by 55 points with the bridges attached — and a fifth lesson closed it: a gate trained only on prompts where the channel should open is not a gate; trained also on prompts where it should be silent, it becomes one, at a cost of two items in a hundred on the task. The same recipe then carried a second family, Gemma 4 12B, to within one item of its oracle in a single pass, once its readout was standardized against a residual-stream property the Qwen backbones do not share — the one adjustment the transfer needed, and one that was measured rather than tuned.

9.6 Multi-mode at 12B: the support-coverage law, restated as a design rule

The multi-mode task of Section 7 was then run on Gemma 4 12B with the released recipe (7,500 steps: 2,000 readout-only, 4,000 coupled, 1,500 no-harm; `results/stage2b_gemma12b_2b/`). Held-out, n=48 per family: in-distribution **100%** (MAE 0.009), the held-out mode combination **25.0%** (0.123), amplitude extrapolation **52.1%** (0.086), against a backbone alone at 20.8%/16.7%/10.4%, an always-zero strategy at 27.1%/16.7%/4.2%, and an oracle at 100% on all three. Against the 0.5B numbers of Section 7 (97.9%, 31.3%, 50.0%) the in-distribution family is now saturated and the two generalization families are unchanged within noise. Twenty-four times the backbone buys the training distribution and nothing outside it: the support-coverage law is scale-independent.

The attribution is sharper here than in Section 7. Running the frozen FNO on each item's *true* parameters — no language model, no bridges — scores 100% on all three families at MAE

0.0002–0.0008, so the physics hemisphere is exact everywhere and the entire loss is the readout's. Two measurements say what the readout does instead. On the combination family, 19 of 48 answers match a *single*-mode field value: one pooled vector regressed to six numbers cannot express "there is a second term here" when every training prompt had one term. On the extrapolation family, inverting each answer to the amplitude that would produce it gives an implied amplitude below the true one for 71% of items at a median ratio of 0.68 — inside mode 1's training range of 0.3–0.7. The regression head clamps to the amplitudes it was shown, exactly as the law predicts, and the v2 refutation of Section 7 says that binning it per mode does not help, because bins outside a mode's trained range never receive gradient.

The law then names its own remedy, and it is not a change of parameterization but a change of *coverage*: read each mode's amplitude with the *same* head. Mode 2 is trained on amplitudes 0.5–1.0, which covers the 0.8–1.0 that mode 1 is tested on, so a shared head has already been taught those digits — in another slot. The structural half is handled the way x_0 was handled: each number is read from its own deterministic token span, so a two-term prompt presents two spans and an absent mode presents an empty one.

We tested this before spending the 12B budget, on a 0.5B backbone and teacher-forced: train only the readout, then score what the physics model would answer from the parameters it produces (`eval/readout_transfer_probe.py`; the pooled readout reproduces the rollout profile of Section 7, 0.99/0.32/0.44, which is what makes the probe usable as a proxy). The shared-head span readout lifts extrapolation from 0.44 to **0.96–0.98** at every readout depth and seed we tried, at an amplitude error of 0.001–0.002 on a range that slot never saw — the coverage prediction, confirmed. The combination family did *not* follow: it ranged 0.30–0.98 with no dependence on anything we varied, and a readout-depth effect we first took for a second design rule turned out to be seed noise (0.979 and 0.396 on two seeds of one configuration). Three seeds per arm then separated two mechanisms. Adding *structure* coverage — rewriting half the single-mode prompts with the absent mode present at a small amplitude drawn from $U(0.02, 0.25)$, the answer recomputed by the solver — takes the combination family from 0.608 ± 0.136 to $\mathbf{0.993 \pm 0.012}$, while a distractor pinned at exactly 0.0, which supplies the structure without anything to read in it, gains almost nothing (0.681 ± 0.054). So the law splits in two: *structure* coverage is cheap and necessary — the prompt shapes must occur in training, though the values in them may be tiny — while *value* coverage is bought by sharing a head across slots rather than by showing the values in place. We note the cost of the remedy in the same breath: with two-term prompts in training, that family is no longer a test of compositional structure but of amplitude range in a two-term context, a real generalization test and a weaker one. The 12B run under this recipe is in progress at the time of writing.

9.7 A leaky gate: what an always-on channel costs, and what it carries

The selective gate of run 9 is shut off-task ($\bar{\sigma} \leq 0.003$), so on a GSM8K prompt the channel carries nothing. Would a *weak* always-on signal act as a regularizer, or improve reasoning by keeping a quantitative model faintly in the loop? The question is cheap to test and the answer turns out to separate two things the architecture had so far only ever varied together: the channel's *presence* and its *content*.

Method. A floor is applied to the gate at inference on the run-9 bridges (`results/stage2_mlx8b9/bridges.npz` step 5000, coupling 15/22 of 36), with nothing retrained: $\sigma_{\text{eff}} = \varepsilon + (1 - \varepsilon)\,\sigma$. The form is monotone in σ and leaves the open end alone, where $\max(\varepsilon, \sigma)$ would flatten the gate's own decision everywhere below the floor. The *pre-floor* σ is what we log, so that decision stays observable. At prompt positions it is identical across arms to five decimals; the all-position mean, which also covers generated tokens and so is taken over different text in different arms, drifts by at most 2×10^{-3}

across the Qwen sweep (physics flat to 6×10^{-5}). On Gemma at $\varepsilon = 1.0$ it is not invariant at all ($0.144 \to 0.117$ on physics): the gate is computed from the residual stream it sits in, so an injection large enough to disturb that stream changes what the gate reads — feedback the pre-floor column exists to expose. Off-task the floor is nearly the whole gate and on-task nearly nothing: at $\varepsilon = 0.2$ it takes σ_{eff} from 0.0016 to 0.201 on GSM8K but from 0.791 to 0.833 at physics prompt positions. Arms join the backbone and ΨLM's trained gate on the guard-rail of Table 7, extended with BoolQ (validation split, yes/no over a passage under an `Answer: <yes/no>` instruction, a 16-token budget), n=100 per dataset at seed 0, greedy throughout, with a per-token KL(base $\|$ arm) over the full vocabulary teacher-forced on the base arm's own greedy continuation (`eval/bench_guardrail.py`, tabulated by `eval/leaky_report.py`). Two properties of the always-on signal matter for reading what follows: the physics tokens are computed once from the prompt and re-injected unchanged at every position, so this is one constant vector per question; and on a non-physics prompt the value it carries is a Burgers field value for whatever initial condition the readout extracted from a word problem — a number, but not a fact about the question.

The dose-response curve. Table 8 gives the sweep from $\varepsilon = 0.01$ to the fully open gate. Nothing off-task is *harmed* until $\varepsilon = 0.5$, where GSM8K loses eight items (89% → 81%; against the backbone nine lost and one gained, $p = 0.022$; against the trained gate ten and two, $p = 0.039$) — the first significant off-task harm. MMLU rises earlier and significantly, from $\varepsilon = 0.3$ ($p = 0.012$ against the backbone); the next paragraph is why that is not a gain. Past that it falls apart: 50% at $\varepsilon = 0.7$, 19% at 0.9, and **8% at** $\varepsilon = 1.0$. That last arm is run 8's operating point reached by force rather than by training, and it settles the question that paragraph left open: the collapse needs only an open gate, not open-gate training. It also refines it. Run 8, whose gate was *trained* open, scored 34% on GSM8K at 70 generated tokens; forcing the same operating point on the selective bridges gives 8% at 24 tokens. Forcing open is four times more destructive than training open, so run 8's system had partially adapted to its own channel, and the no-harm arm's contribution was to close the gate rather than to repair damage. MMLU replicates that paragraph's finding exactly: 69% at 7 generated tokens in both.

The MMLU rise is a token budget, not reasoning. It is the one off-task column that rises, and it rises with three others: mean reply length falls $87 \to 62 \to 7$ tokens across the sweep, the EOS rate climbs $0.71 \to 0.82$, and the parse rate climbs $0.72 \to 0.84 \to 1.00$. A shorter reply finishes inside the 256-token budget and gets scored. Of the 28 items the backbone leaves unparsed, $\varepsilon = 0.2$ parses 13 and loses 1, all 13 in the two subjects that carry the whole movement (college physics, mathematics), and 7 are right — 54%, against the 75% the backbone scores on the items it already parses in those subjects. The extra points are items scored, not items solved. On the 71 questions *every* arm answered, accuracy is 83.1% in all six arms including the backbone, though not item-identical: each coupled arm loses one and gains one. That control is weaker than the identity looks — 60 of its 71 items are philosophy, biology and foreign policy, where every arm emits four tokens and parses — so it shows the floor changes nothing where length was never binding without settling the items where it was. GSM8K and BoolQ close it from the other side: both parse at 1.000 in every arm, the same brevity appears on GSM8K ($147 \to 137$ tokens at $\varepsilon = 0.2$), and accuracy does not move.

The safe range is a property of the backbone, not a constant. Rerunning the sweep on Gemma 4 12B (`results/bench/leaky_gemma_guardrail_summary.json`; the same protocol, floors chosen around *its* operating point) puts the boundary four to eight times lower: unharmed at $\varepsilon = 0.05$, wobbling at 0.1 (physics 91%, $p = 0.03$; GSM8K 71%, $p = 0.004$), destroyed at 0.2 (physics 50%, GSM8K 13%, all $p < 10^{-3}$), and at $\varepsilon = 1.0$ producing nothing at all on any dataset, at a KL to base of 15–17 where the Qwen 8B's was 0.84. Most of that gap is explained by where each gate was trained to sit: Qwen's is 0.791 on physics against Gemma's 0.144, so the same floor pushes them to very different multiples of their own operating point. In ε the two backbones differ by a factor of eight; in $\sigma_{\text{eff}}/\sigma_{\text{trained}}$ they differ by about 1.4 (Qwen breaks GSM8K at 1.13× and is destroyed by 1.26×; Gemma is intact at 1.30×, breaks at 1.59×, destroyed at 2.18×). We do not claim a law: closing the remaining gap needs the realized injection-to-stream ratio, which `return_ratio` in `psilm/mlx/bridges.py` can record and these sweeps did not. The MMLU budget effect does not replicate on Gemma, and the reason is the width of the window rather than the sign of the effect. Inside Gemma's safe range the floor barely moves reply length at all (MMLU 134.0 and 134.1 tokens at $\varepsilon = 0.02$ and 0.05, against the trained gate's 134.4), so there is no regime in which it shortens replies without destroying the model; Gemma's MMLU accuracy never rises above 56%. The lengthening one sees at larger floors (158.6 tokens at $\varepsilon = 0.2$, 162.5 at 1.0) belongs to arms already collapsed to 37% and 0%, and is not even monotone — GSM8K falls back to 194 tokens at $\varepsilon = 1.0$. It is derailment, not brevity with the opposite sign.

A content control, and a double dissociation. If the brevity and the MMLU rise were the physics model's *output* doing the work, corrupting that output should remove them. The `shuffled`ε arms are `leaky`ε with exactly one substitution: the value encoder is fed the $\hat{u}$ of a *different* question from the same dataset, taken from the first sweep's own recorded readouts by a one-step rotation of the sorted question ids — 400 substitutions, zero fixed points, and the multiset of injected values unchanged, so magnitude is matched by construction. The readout still runs on the real prompt and the gate still decides; `diag.u_injected` records what entered the channel beside the $\hat{u}$ that was computed.

Off-task, the swap changes nothing. At $\varepsilon = 0.2$, MMLU is 0.66 against 0.66 (61 tokens against 62, parse 0.850 against 0.840), GSM8K 0.88 against 0.88 (136 tokens against 137), BoolQ 0.87 against 0.87; every paired test gives $p = 1.00$, and at $\varepsilon = 1.0$ the GSM8K collapse is reproduced at 0.07 against 0.08. Even the KL is indistinguishable (GSM8K 0.055 against 0.058). **The brevity, the MMLU gain and the collapse are content-independent**: any value of that magnitude arriving through the channel produces them.

On-task, the same swap is annihilating: physics falls from 98% to **9%** ($p < 10^{-4}$, MAE $0.018 \to 0.410$) at identical token counts and parse rates, and its KL to base is unchanged at 0.222 — the output distribution moves exactly as far, to a different place. The mechanism is visible per item: the spoken answer sits 0.0121 from the *injected* value and within tolerance of it on **99 of 100** questions, while sitting 0.410 from the truth and from the readout's own $\hat{u}$. Told -0.546 for a question whose answer is -0.095, the model says -0.550.

This is the cleanest evidence in this work that the frozen model reads the latent channel rather than a template or a correlate. The zeroed arm of Table 6 shows that removing the injection removes the physics result; the shuffled arm shows that corrupting *only the number*, leaving length, parsing, gate and off-task behaviour byte-comparable, redirects the answer to the corrupted value with 99% fidelity. Content and presence dissociate: presence explains everything the channel does off-task, content explains everything it does on-task.

What this says. A weak always-on physics signal is not a regularizer and does not improve reasoning: what it improves is the chance of finishing inside a token budget, and that is worth reporting as a decoding effect rather than a capability. The claim is about inference — these bridges were trained with a zero floor and a no-harm arm that penalized exactly the gate this sweep forces open, so nothing ever asked the channel to carry something useful off-task, and a system *co-trained* with an always-on channel is untested. What the sweep does establish is a cost bound and a mechanism: up to $\varepsilon = 0.2$ — at most 4% of the residual stream — the coupled model is never more than one item below its backbone off-task (GSM8K −1, BoolQ −1); the first significant loss is GSM8K at $\varepsilon = 0.5$, roughly a tenth of the stream. Within that range everything the channel does off-task is caused by its presence rather than its content, and everything it does on-task by its

Table 8: Leaky-gate sweep on the run-9 8B bridges: a floor $\sigma_{\text{eff}} = \varepsilon + (1-\varepsilon)\sigma$ applied at inference, nothing retrained. n=100 per dataset, greedy, seed 0; McNemar against the trained gate. `shuffled` arms are the content control (another question's value at the same floor). Mean generated tokens in parentheses where they carry the mechanism. Records: `results/bench/leaky_8b{,_hi,_top,_shuf}_guardrail_summary.json`.

arm	physics	MMLU	GSM8K	BoolQ	physics MAE
backbone alone	2%	60% (87)	89% (147)	88%	2.239
trained gate	95%	61% (83)	89% (145)	88%	0.0215
$\varepsilon = 0.05$	96%	63% (77)	88%	88%	0.0214
$\varepsilon = 0.1$	96%	66% (67)	89%	88%	0.0204
$\varepsilon = 0.2$	98%	66% (62)	88% (137)	87%	**0.0182**
$\varepsilon = 0.3$	97%	69% (54)	88%	85%	0.0189
$\varepsilon = 0.5$	94%	70% (24)	**81%*** (108)	84%	0.0202
$\varepsilon = 0.7$	90%	69% (10)	**50%**** (74)	77%**	0.0219
$\varepsilon = 1.0$	89%	69% (7)	**8%**** (24)	59%**	0.0236
`shuffled` 0.2	**9%****	66% (61)	88% (136)	87%	0.4098
`shuffled` 1.0	**9%****	70% (14)	7% (28)	64%**	0.4083

$^{*}\,p < 0.05$, $^{**}\,p < 0.01$ against the trained gate.

content alone.

10 Discussion and Limitations

What has been shown. A frozen 0.5B language model and frozen partner models of three kinds (a twin LLM driving a calculator; a purpose-trained FNO; pretrained DPOT-Tiny) can be composed into systems that answer questions none of the parts can answer, with all inter-model traffic in latent space and only 2–7M trained parameters. At this scale the latent channel matches text-level oracle accuracy; its digit-level fidelity (Stage 1's long products) is bounded by interface capacity and training budget, consistent with Flamant et al. [12].

The strongest evidence that the traffic is what we say it is comes from a control run late: corrupting *only* the number in the channel — another question's value, at matched magnitude, with length, parsing, gate and off-task behaviour unchanged — redirects the frozen model's answer to the corrupted value on 99 of 100 held-out questions (Section 9.7). The coupled model is reading the channel, not a template, a prompt correlate, or the reply format.

What has not been shown. The forward readouts are parametric: the interface extracts a fixed set of quantities the task designer chose, and Section 7 shows transfer beyond their trained support is partial. General language-to-field translation — arbitrary ICs described in free text — remains open. Scaling the language side (Section 9) reaches 1.7B cleanly and 8B only by supplying the pointer from the task and narrowing the channel to the queried value: where Stage 0 predicts larger gains in evidence use, what we measured at 8B was first larger *costs* in interface optimization and then, once the interface was narrow enough, an oracle-level result — a gain in evidence use over the 0.5B and 1.7B systems has not been demonstrated, because the task saturates. Harmlessness off-task has been shown for the 8B on GSM8K, MMLU and BoolQ (Table 7 and Section 9.7, which also bounds the cost of an always-on channel and separates what that channel does by its presence from what it does by its content) and not measured for the 0.5B and 1.7B systems, whose gates were likewise trained on physics prompts only (the 1.7B's ended its run at 1.0, the 0.5B's at a mean of 0.13 over all positions). Our tasks are also ones where the oracle-text arm saturates; the deeper promise of latent coupling — carrying information that does not survive verbalization, at bandwidth text cannot match — is precisely what these tasks cannot yet measure. The brain analogy and the complementarity framing are motivations, not claims; likewise the author's broader conjecture that grounding language models in physical law may support alignment is a direction for argument, not something these experiments establish.

Why small-scale results matter here. Every mechanism-level finding in this paper — the capability-gap reversal, the phase transitions and their causal order, the exposure-bias bootstrap, the mode collapse of unpointed attention, the support-coverage law — was discovered by failure analysis cheap enough to run on a laptop, and each transferred forward to the next, more capable configuration. The same interfaces are, by construction, what a frontier-scale coupling would train.

11 Reproducibility

Everything runs on one Apple M2 (24 GB). The repository `https://github.com/ryoji-info/PsiLM` contains all code, seeded data generators, committed QA datasets, training logs, and evaluation records, including the failed designs; each stage reproduces with two or three commands documented in the README. Total training compute across all experiments is roughly 30 hours of Apple-silicon time.

12 AI Generation Disclosure

This work was generated by **Claude Fable 5** (Anthropic), operating as an autonomous research agent within Claude Code on the author's laptop between August 28 and September 5, 2026, under the author's direction and review. Fable 5 performed the literature survey, proposed and implemented all architectures, ran and monitored all training, diagnosed the failures, formulated and refuted the intermediate hypotheses, and wrote this manuscript, including this section. The human author set the research question, the guiding analogies, the hardware constraint, and the project's name, approved each stage, and bears responsibility for the published claims. All quantitative results are taken directly from committed experimental logs in the repository; the bibliography contains only works surfaced and verified during the project's literature searches.

Acknowledgments

The Stage 1 design follows the published specification of Flamant et al. [12]; DPOT-Tiny is due to Hao et al. [17] (Apache-2.0). This project was conducted independently on consumer hardware.

References

[1] Jean-Baptiste Alayrac et al. Flamingo: a visual language model for few-shot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2204.14198.

[2] Rachit Bansal et al. LLM augmented LLMs: Expanding capabilities through composition. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2401.02412.

[3] Kristian G. Barman et al. Large physics models: Towards a collaborative approach with large language models and foundation models. *European Physical Journal C*, 85, 2025. arXiv:2501.05382.

[4] Gabriel Béna and Dan F. M. Goodman. Dynamics of specialization in neural modules under resource constraints. *Nature Communications*, 15, 2024. arXiv:2106.02626.

[5] Johan Bjorck et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.

[6] Grady Booch, Francesco Fabiano, Lior Horesh, et al. Thinking fast and slow in AI. In *AAAI Conference on Artificial Intelligence*, 2021. arXiv:2010.06002.

[7] Konstantina Christakopoulou, Shibl Mourad, and Maja Matarić. Agents thinking fast and slow: A Talker-Reasoner architecture. *arXiv preprint arXiv:2410.08328*, 2024.

[8] Benjamin Devillers, Léopold Maytié, and Rufin VanRullen. Semi-supervised multimodal representation learning through a global workspace. *IEEE Transactions on Neural Networks and Learning Systems*, 2024. arXiv:2306.15711.

[9] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2305.14325.

[10] FAIR Diplomacy Team, Anton Bakhtin, Noam Brown, et al. Human-level play in the game of Diplomacy by combining language models with strategic reasoning. *Science*, 378(6624), 2022.

[11] Figure AI. Helix: A vision-language-action model for generalist humanoid control. Company research release, `https://www.figure.ai/news/helix`, 2025.

[12] Cedric Flamant, Udaya Ghai, and Kanna Shimizu. The Bicameral Model: Bidirectional hidden-state coupling between parallel language models. *arXiv preprint arXiv:2605.11167*, 2026.

[13] Gemma Team and Google DeepMind. Gemma 4. Model card and technical report, `https://huggingface.co/google/gemma-4-12B-it`, 2026. 4-bit MLX conversion: `https://huggingface.co/mlx-community/gemma-4-12B-it-4bit`.

[14] Anirudh Goyal, Aniket Didolkar, Alex Lamb, et al. Coordination among neural modules through a shared global workspace. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2103.01197.

[15] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on apple silicon. `https://github.com/ml-explore/mlx`, 2023.

[16] Shibo Hao, Yi Gu, Haodi Ma, et al. Reasoning with language model is planning with world model. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2023. arXiv:2305.14992.

[17] Zhongkai Hao et al. DPOT: Auto-regressive denoising operator transformer for large-scale PDE pre-training. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2403.03542.

[18] Zhiting Hu and Tianmin Shu. Language models, agent models, and world models: The LAW for machine reasoning and planning. *arXiv preprint arXiv:2312.05230*, 2023.

[19] Yann LeCun. A path towards autonomous machine intelligence. OpenReview position paper, version 0.9.2, 2022.

[20] Junnan Li et al. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2301.12597.

[21] Ruibo Liu et al. Mind's Eye: Grounded language model reasoning through simulation. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.05359.

[22] Xiaoze Liu et al. The Vision Wormhole: Latent-space communication in heterogeneous multi-agent systems. *arXiv preprint arXiv:2602.15382*, 2026.

[23] Yingzhuo Liu. Beyond tokens: A unified framework for latent communication in LLM-based multi-agent systems. *arXiv preprint arXiv:2606.05711*, 2026.

[24] Yuhan Liu et al. DroidSpeak: KV cache sharing for cross-LLM communication and multi-LLM serving. *arXiv preprint arXiv:2411.02820*, 2024.

[25] Pingchuan Ma et al. LLM and simulation as bilevel optimizers: A new paradigm to advance physical scientific discovery. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2405.09783.

[26] Léopold Maytié, Roland Bertin Johannet, and Rufin VanRullen. Multimodal dreaming: A global workspace approach to world model-based reinforcement learning. *arXiv preprint arXiv:2502.21142*, 2025.

[27] Elisa Negrini, Yuxuan Liu, Liu Yang, Stanley J. Osher, and Hayden Schaeffer. A multi-modal PDE foundation model for prediction and scientific text descriptions. *arXiv preprint arXiv:2502.06026*, 2025.

[28] Chau Pham et al. Let models speak ciphers: Multiagent debate through embeddings. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06272.

[29] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[30] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[31] Chandramouli Rajagopalan, David Rawlinson, Elkhonon Goldberg, and Gideon Kowadlo. Deep learning in a bilateral brain with hemispheric specialization. *arXiv preprint arXiv:2209.06862*, 2022.

[32] Vignav Ramesh and Kenneth Li. Communicating activations between language model agents. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2501.14082.

[33] Jarrad Rinaldo, Levin Kuhlmann, Jason Friedman, and Gideon Kowadlo. Left/right brain, human motor control and the implications for robotics. *arXiv preprint arXiv:2401.14057*, 2024.

[34] Gleb Rodionov et al. Hogwild! Inference: Parallel LLM generation via concurrent attention. *arXiv preprint arXiv:2504.06261*, 2025.

[35] Junhong Shen, Tanya Marwah, and Ameet Talwalkar. UPS: Efficiently building foundation models for PDE solving via cross-modal adaptation. *arXiv preprint arXiv:2403.07187*, 2024.

[36] Shannon Zejiang Shen, Hunter Lang, Bailin Wang, Yoon Kim, and David Sontag. Learning to decode collaboratively with multiple language models. In *Association for Computational Linguistics (ACL)*, 2024. arXiv:2403.03870.

[37] Zhengqi Sun et al. Reason–Imagine–Act: Closed-loop LLM decision making with world models for autonomous driving. *arXiv preprint arXiv:2605.24004*, 2026.

[38] Rufin VanRullen and Ryota Kanai. Deep learning and the global workspace theory. *Trends in Neurosciences*, 44(9), 2021. arXiv:2012.10390.

[39] Sumanth Varambally et al. Zephyrus: An agentic framework for weather science. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.04017.

[40] Ziqi Wang, Boye Niu, et al. A survey on parallel reasoning. *arXiv preprint arXiv:2510.12164*, 2025.

[41] Liu Yang, Siting Liu, and Stanley J. Osher. Fine-tune language models as multi-modal differential equation solvers. *Neural Networks*, 2025. arXiv:2308.05061.

[42] Andy Zeng et al. Socratic models: Composing zero-shot multimodal reasoning with language. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2204.00598.

[43] Anthony Zhou, Zijie Li, Michael Schneier, John R. Buchanan Jr, and Amir Barati Farimani. Text2PDE: Latent diffusion models for accessible physics simulation. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.01153.

[44] Hang Zhou et al. Unisolver: PDE-conditional transformers towards universal neural PDE solvers. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2405.17527.

[45] Konrad Zolna, Serkan Cabi, Yutian Chen, et al. GATS: Gather-Attend-Scatter. *arXiv preprint arXiv:2401.08525*, 2024.

[46] Jiaru Zou et al. Latent collaboration in multi-agent systems. *arXiv preprint arXiv:2511.20639*, 2025.